

Sesión 14 · Experimenta

10 ejercicios progresivos de struct y arreglos de struct en C++

Curso: Principios de Programación

Nivel: 1.er ciclo

Entorno sugerido: Code::Blocks

Regla de diseño: solo contenidos vistos hasta la sesión 14

Progresivo

1–4: un solo struct

5–8: arreglos de struct

9: ordenamiento simple

10: mini sistema con menú

Ruta de aprendizaje

De una ficha simple a una mini base de datos en memoria

1. Perfil gamer
2. Ficha estudiante
3. Pedido cafeteria
4. Atleta universitario
5. Playlist del grupo
6. Gastos de semana
7. Ranking de videojuegos
8. Buscar contacto por codigo
9. Ordenar jugadores por puntos
10. Mini sistema de evento

Ejercicio 1: Perfil gamer

Contexto

Un estudiante quiere registrar la ficha de su videojuego favorito.

Enunciado

Crea un struct Videojuego con titulo, genero, horasJugadas y precio.
Declara una variable, asigna datos directamente y muestra la ficha completa.

Solución C++ · Ejercicio 1: Perfil gamer

```
#include <iostream>
#include <string>
using namespace std;

struct Videojuego {
    string titulo;
    string genero;
    int horasJugadas;
    double precio;
};

int main() {
    Videojuego favorito;

    favorito.titulo = "Rocket League";
    favorito.genero = "Deportes";
    favorito.horasJugadas = 120;
    favorito.precio = 0.00;

    cout << "--- FICHA DEL VIDEOJUEGO ---" << endl;
    cout << "Titulo: " << favorito.titulo << endl;
    cout << "Genero: " << favorito.genero << endl;
    cout << "Horas jugadas: " << favorito.horasJugadas << endl;
    cout << "Precio: S/ " << favorito.precio << endl;

    return 0;
}
```

Ejercicio 2: Ficha estudiante

Contexto

El tutor del aula necesita registrar una ficha básica de un estudiante de primer ciclo.

Enunciado

Crea un struct Estudiante con nombre, edad y promedio.
Solicita los datos al usuario y muestra la ficha registrada.

Solución C++ · Ejercicio 2: Ficha estudiante

```
#include <iostream>
#include <string>
using namespace std;

struct Estudiante {
    string nombre;
    int edad;
    double promedio;
};

int main() {
    Estudiante e;

    cout << "Nombre del estudiante: ";
    cin >> e.nombre; // Se usa cin porque el ejemplo espera un nombre sin espacios.

    cout << "Edad: ";
    cin >> e.edad;

    cout << "Promedio: ";
    cin >> e.promedio;

    cout << "\n--- FICHA REGISTRADA ---" << endl;
    cout << "Nombre: " << e.nombre << endl;
    cout << "Edad: " << e.edad << endl;
    cout << "Promedio: " << e.promedio << endl;

    return 0;
}
```

Ejercicio 3: Pedido cafeteria

Contexto

Un grupo de amigos compra snacks en la cafetería de la universidad.

Enunciado

Crea un struct Producto con nombre, precioUnitario y cantidad.
Pide los datos, calcula el total a pagar y muestra un resumen del pedido.

Solución C++ · Ejercicio 3: Pedido cafeteria

```
#include <iostream>
#include <string>
using namespace std;

struct Producto {
    string nombre;
    double precioUnitario;
    int cantidad;
};

int main() {
    Producto p;
    double total;

    cout << "Producto comprado: ";
    cin >> p.nombre;

    cout << "Precio unitario: S/ ";
    cin >> p.precioUnitario;

    cout << "Cantidad: ";
    cin >> p.cantidad;

    total = p.precioUnitario * p.cantidad;

    cout << "\n--- RESUMEN DEL PEDIDO ---" << endl;
    cout << "Producto: " << p.nombre << endl;
    cout << "Precio unitario: S/ " << p.precioUnitario << endl;
    cout << "Cantidad: " << p.cantidad << endl;
    cout << "Total a pagar: S/ " << total << endl;

    return 0;
}
```


Ejercicio 4: Atleta universitario

Contexto

En una competencia universitaria de 100 metros se registra el tiempo de un participante.

Enunciado

Crea un struct Atleta con nombre y tiempo.
Pide los datos y muestra si su marca es sobresaliente si el tiempo es menor o igual a 12 segundos.

Solución C++ · Ejercicio 4: Atleta universitario

```
#include <iostream>
#include <string>
using namespace std;

struct Atleta {
    string nombre;
    double tiempo;
};

int main() {
    Atleta a;

    cout << "Nombre del atleta: ";
    cin >> a.nombre;

    cout << "Tiempo en 100 metros (segundos): ";
    cin >> a.tiempo;

    cout << "\nResultado de " << a.nombre << ": " << a.tiempo << " segundos." << endl;

    if (a.tiempo <= 12.0) {
        cout << "Marca sobresaliente para la competencia universitaria." << endl;
    } else {
        cout << "Marca registrada. Puede seguir entrenando para mejorar." << endl;
    }

    return 0;
}
```

Ejercicio 5: Playlist del grupo

Contexto

Un grupo de amigos arma una playlist para una reunión universitaria.

Enunciado

Crea un struct Cancion con codigo y titulo.
Registra 4 canciones en un arreglo de structs y muestra la playlist completa.

Solución C++ · Ejercicio 5: Playlist del grupo

```
#include <iostream>
#include <string>
using namespace std;

struct Cancion {
    int codigo;
    string titulo;
};

int main() {
    const int SIZE = 4;
    Cancion playlist[SIZE];

    for (int i = 0; i < SIZE; i++) {
        cout << "Cancion " << i + 1 << " - codigo: ";
        cin >> playlist[i].codigo;

        cout << "Cancion " << i + 1 << " - titulo: ";
        cin >> playlist[i].titulo;
    }

    cout << "\n--- PLAYLIST REGISTRADA ---" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << playlist[i].codigo << " - " << playlist[i].titulo << endl;
    }

    return 0;
}
```

Ejercicio 6: Gastos de semana

Contexto

Un estudiante quiere controlar sus gastos de lunes a viernes.

Enunciado

Crea un struct Gasto con descripcion y monto.
Registra 5 gastos en un arreglo, acumula el total y muestra el gasto promedio.

Solución C++ · Ejercicio 6: Gastos de semana

```
#include <iostream>
#include <string>
using namespace std;

struct Gasto {
    string descripcion;
    double monto;
};

int main() {
    const int SIZE = 5;
    Gasto gastos[SIZE];
    double total = 0;
    double promedio;

    for (int i = 0; i < SIZE; i++) {
        cout << "Gasto " << i + 1 << " descripcion: ";
        cin >> gastos[i].descripcion;

        cout << "Gasto " << i + 1 << " monto: S/ ";
        cin >> gastos[i].monto;

        total += gastos[i].monto; // Acumulador: suma todos los montos.
    }

    promedio = total / SIZE;

    cout << "\n--- REPORTE DE GASTOS ---" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << gastos[i].descripcion << ": S/ " << gastos[i].monto << endl;
    }

    cout << "Total gastado: S/ " << total << endl;
    cout << "Promedio por gasto: S/ " << promedio << endl;

    return 0;
}
```

Ejercicio 7: Ranking de videojuegos

Contexto

Tres amigos comparan videojuegos y quieren saber cuál tiene la mayor calificación.

Enunciado

Crea un struct Juego con titulo y calificacion.
Registra 3 juegos en un arreglo y muestra cuál tiene la mayor calificación.

Solución C++ · Ejercicio 7: Ranking de videojuegos

```
#include <iostream>
#include <string>
using namespace std;

struct Juego {
    string titulo;
    double calificacion;
};

int main() {
    const int SIZE = 3;
    Juego juegos[SIZE];
    int posicionMayor = 0; // Suponemos inicialmente que el primero es el mejor.

    for (int i = 0; i < SIZE; i++) {
        cout << "Juego " << i + 1 << " titulo: ";
        cin >> juegos[i].titulo;

        cout << "Calificacion (0 a 20): ";
        cin >> juegos[i].calificacion;
    }

    for (int i = 1; i < SIZE; i++) {
        if (juegos[i].calificacion > juegos[posicionMayor].calificacion) {
            posicionMayor = i;
        }
    }

    cout << "\nEl juego mejor calificado es: " << juegos[posicionMayor].titulo << endl;
    cout << "Calificacion: " << juegos[posicionMayor].calificacion << endl;

    return 0;
}
```


Ejercicio 8: Buscar contacto por código

Contexto

Un estudiante registra contactos de un grupo de trabajo y luego busca uno por código.

Enunciado

Crea un struct Contacto con código y nombre.
Registra 5 contactos en un arreglo y busca un contacto por código usando búsqueda lineal.

Solución C++ · Ejercicio 8: Buscar contacto por código

```
#include <iostream>
#include <string>
using namespace std;

struct Contacto {
    int codigo;
    string nombre;
};

int main() {
    const int SIZE = 5;
    Contacto contactos[SIZE];
    int codigoBuscado;
    int posicion = -1; // -1 indica que todavía no se encontró el contacto.

    for (int i = 0; i < SIZE; i++) {
        cout << "Contacto " << i + 1 << " código: ";
        cin >> contactos[i].codigo;

        cout << "Contacto " << i + 1 << " nombre: ";
        cin >> contactos[i].nombre;
    }

    cout << "\nIngrese el código a buscar: ";
    cin >> codigoBuscado;

    for (int i = 0; i < SIZE; i++) {
        if (contactos[i].codigo == codigoBuscado) {
            posicion = i;
            break; // Se usa break porque ya encontramos el contacto.
        }
    }

    if (posicion == -1) {
        cout << "Contacto no encontrado." << endl;
    } else {
        cout << "Contacto encontrado: " << contactos[posicion].nombre << endl;
    }

    return 0;
}
```

Ejercicio 9: Ordenar jugadores por puntos

Contexto

Después de un torneo gamer, se quiere ordenar a los jugadores por puntaje de mayor a menor.

Enunciado

Crea un struct Jugador con nombre y puntos.
Registra 4 jugadores y ordénalos de mayor a menor usando selección.
Muestra el ranking final.

Solución C++ · Ejercicio 9: Ordenar jugadores por puntos (1/2)

```
#include <iostream>
#include <string>
using namespace std;

struct Jugador {
    string nombre;
    int puntos;
};

int main() {
    const int SIZE = 4;
    Jugador jugadores[SIZE];

    for (int i = 0; i < SIZE; i++) {
        cout << "Jugador " << i + 1 << " nombre: ";
        cin >> jugadores[i].nombre;

        cout << "Puntos: ";
        cin >> jugadores[i].puntos;
    }

    // Ordenamiento por seleccion descendente.
    // En cada vuelta buscamos al jugador con mayor puntaje restante.
    for (int start = 0; start < SIZE - 1; start++) {
        int posMayor = start;

        for (int i = start + 1; i < SIZE; i++) {
            if (jugadores[i].puntos > jugadores[posMayor].puntos) {
                posMayor = i;
            }
        }

        Jugador temp = jugadores[start];
        jugadores[start] = jugadores[posMayor];
    }
}
```

Solución C++ · Ejercicio 9: Ordenar jugadores por puntos (2/2)

```
        jugadores[posMayor] = temp;
    }

    cout << "\n--- RANKING FINAL ---" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << i + 1 << ". " << jugadores[i].nombre
            << " - " << jugadores[i].puntos << " puntos" << endl;
    }

    return 0;
}
```

Ejercicio 10: Mini sistema de evento

Contexto

Un comité de estudiantes organiza un evento universitario y necesita registrar asistentes.

Enunciado

Crea un struct Asistente con codigo, nombre y edad.
Implementa un menú con do-while para registrar hasta 5 asistentes, listar asistentes, buscar por código y salir.

Solución C++ · Ejercicio 10: Mini sistema de evento (1/3)

```
#include <iostream>
#include <string>
using namespace std;

struct Asistente {
    int codigo;
    string nombre;
    int edad;
};

int main() {
    const int MAX = 5;
    Asistente asistentes[MAX];
    int cantidad = 0;
    int opcion;

    do {
        cout << "\n--- MENU EVENTO UNIVERSITARIO ---" << endl;
        cout << "1. Registrar asistente" << endl;
        cout << "2. Listar asistentes" << endl;
        cout << "3. Buscar asistente por codigo" << endl;
        cout << "4. Salir" << endl;
        cout << "Opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                if (cantidad < MAX) {
                    cout << "Codigo: ";
                    cin >> asistentes[cantidad].codigo;

                    cout << "Nombre: ";
                    cin >> asistentes[cantidad].nombre;
                }
            }
        }
    } while (opcion != 4);
}
```

Solución C++ · Ejercicio 10: Mini sistema de evento (2/3)

```
        cout << "Edad: ";
        cin >> asistentes[cantidad].edad;

        cantidad++;
        cout << "Asistente registrado." << endl;
    } else {
        cout << "Ya no hay espacio para mas asistentes." << endl;
    }
    break;

case 2:
    if (cantidad == 0) {
        cout << "No hay asistentes registrados." << endl;
    } else {
        cout << "\n--- LISTA DE ASISTENTES ---" << endl;
        for (int i = 0; i < cantidad; i++) {
            cout << asistentes[i].codigo << " - "
                << asistentes[i].nombre << " - "
                << asistentes[i].edad << " años" << endl;
        }
    }
    break;

case 3: {
    int buscado;
    int posicion = -1;

    cout << "Codigo a buscar: ";
    cin >> buscado;

    for (int i = 0; i < cantidad; i++) {
        if (asistentes[i].codigo == buscado) {
            posicion = i;
            break;
        }
    }
}
```


Solución C++ · Ejercicio 10: Mini sistema de evento (3/3)

```
        }  
    }  
  
    if (posicion == -1) {  
        cout << "Asistente no encontrado." << endl;  
    } else {  
        cout << "Encontrado: " << asistentes[posicion].nombre  
            << ", " << asistentes[posicion].edad << " años" << endl;  
    }  
    break;  
}  
  
case 4:  
    cout << "Saliendo del sistema." << endl;  
    break;  
  
default:  
    cout << "Opcion invalida." << endl;  
}  
} while (opcion != 4);  
  
return 0;  
}
```